

00 - Master Build Brief

Visor DOM-to-Agent Context Compiler
DOC-00 | 0.1 requirements pack | Generated 2026-06-14

This master brief is the first document the AI coding agent should read. It defines the project mission, MVP boundary, build order, architecture, non-negotiables, and the initial implementation prompt.

Field	Value
Owner	Rohan PX / Visor project
Audience	AI coding agent, reviewer, future contributors
Status	Draft baseline for MVP build
Decision rule	MVP is local-first, minimal-permission, Chrome Manifest V3, no backend unless explicitly enabled.

Document control

Item	Value
Project	Visor DOM-to-Agent Context Compiler
Document code	DOC-00
Generated on	2026-06-14
Primary goal	Turn raw webpage DOM into safe, structured, agent-ready context.
MVP boundary	Chrome extension, local-only compiler, JSON/Markdown export, privacy redaction, tests.

1. Build mission

Visor converts the current webpage into a structured context object that an AI agent can use safely. The product is not a scraper dump. It is a compiler pipeline: raw DOM -> page snapshot -> normalized content -> privacy-checked context -> agent-ready export.

2. MVP non-negotiables

- Chrome-first, Manifest V3 extension.
- TypeScript-first codebase with strict types.
- Local-first processing; no backend or external AI calls in MVP.
- Minimal permissions: prefer activeTab plus scripting/storage where possible; host permissions only when needed.
- Every extracted webpage instruction is treated as untrusted data, never as developer/system instruction.
- User must preview context before export or external sharing.
- Implementation must include tests for extraction, compiler, privacy redaction, and export formatting.

3. Product boundary

Area	In MVP	Explicitly out of MVP
Extension	Popup, content script, service worker, settings, local storage.	Marketplace, cloud accounts, billing, organization admin.
Compiler	Rule-based extraction, noise removal, token estimate, schema output.	Fully autonomous browser control, remote LLM summarization.
Security	Local redaction, prompt-injection labeling, consent-based export.	Enterprise policy console, advanced DLP integrations.
Data	Local settings, optional local history metadata.	Cloud sync of full page content by default.

4. MVP feature list

- Compile the active tab into a clean AgentContext JSON object.
- Extract visible text, headings, links, buttons, forms, tables, metadata, and code blocks.
- Remove obvious noise: hidden nodes, scripts, styles, boilerplate, cookie banners, repeated nav/footer text.
- Estimate token usage and support compact, detailed, action, RAG, and debug modes.
- Redact obvious sensitive content and warn on risky page classes.
- Preview before export and copy JSON, Markdown, or prompt block output.
- Run local-first with no external AI calls in MVP.

5. System architecture

Layer	Responsibility	Main artifacts
Browser extension shell	User entry point, permissions, popup, options page, service worker, content script injection.	manifest.json, popup.tsx, service-worker.ts, content-script.ts
DOM extraction engine	Capture page structure and visible semantic blocks without executing untrusted content.	extractors/*, visibility.ts, selectors.ts
Intermediate snapshot	Stable internal page model before compression or redaction.	PageSnapshot types, Zod validators
Compiler pipeline	Normalize, dedupe, score, classify, chunk, compress, and format.	compiler/*, tokenBudget.ts
Privacy engine	Detect sensitive data, apply redaction policies, and produce warnings.	privacy/*, threatRules.ts
UI and export layer	Preview, debug, settings, site profiles, copy/export actions.	popup, preview, settings, templates

6. Build order for AI agent

Order	Task	Definition of done
1	Create MV3 extension scaffold	Build runs; extension loads unpacked in Chrome; popup opens.
2	Implement message passing	Popup can request active tab extraction and receive response.
3	Implement basic DOM extractor	URL, title, metadata, headings, text blocks, links are extracted.
4	Add semantic extractors	Buttons, forms, tables, media, code blocks are structured.
5	Define schemas	PageSnapshot and AgentContext validators exist and reject malformed output.
6	Build compiler	Noise filtering, dedupe, scoring, chunking, token estimate, export modes.
7	Add privacy engine	Redaction and risk warnings are present and tested.
8	Build preview/export UI	User can inspect, copy JSON, copy Markdown, view token profile.
9	Add settings/site profiles	Privacy level, token budget, selector overrides, debug mode.
10	Test and package	Unit tests, fixture pages, README, build artifact.

7. First AI coding-agent prompt

You are building a production-quality Chrome Manifest V3 extension called Visor DOM-to-Agent Context Compiler.

Goal: Convert the current webpage DOM into clean, structured, privacy-safe, agent-ready context.

MVP only:

- TypeScript, React, Vite, Manifest V3.
- Content script extracts URL, title, meta description, headings, visible text, links, buttons, forms, tables, and code blocks.
- Service worker coordinates messages.
- Compiler removes noise, deduplicates text, estimates tokens, and outputs AgentContext JSON.
- Privacy redactor detects emails, phones, password fields, API-key-like strings, and JWT-like strings.
- Popup has Compile Page, JSON preview, Copy JSON, and Copy Markdown.
- No backend. No external AI calls. Minimal permissions. Local-first.

Return folder structure, complete code, build instructions, testing instructions, and architecture explanation.

8. Master acceptance gate

- A clean unpacked extension can be installed locally.

- Compile Page works on article, docs, product, dashboard, form, and table-heavy test pages.
- Output is valid AgentContext JSON and includes token profile and privacy report.
- Sensitive obvious values are redacted or warned.
- No network request is made during MVP compilation unless user explicitly enables one in a future mode.
- All requirement IDs in the SRS are mapped to tests or implementation notes.

Source and reference notes

These sources are used to ground platform/security assumptions. The implementation must still verify current platform behavior during development.

Reference	Why it matters
Chrome Extensions - Manifest V3 and service workers: https://developer.chrome.com/docs/extensions/	Grounding for MV3, background service worker model, permissions, and packaging.
MDN WebExtensions - Content scripts: https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/Content_scripts	Grounding for content script behavior, host permissions, and page DOM access.
MDN WebExtensions - storage API: https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/storage	Grounding for extension storage concepts and permissions.
OWASP Browser Extension Vulnerabilities Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Browser_Extension_Vulnerabilities_Cheat_Sheet.html	Grounding for permissions overreach, data leakage, XSS, and insecure communication mitigations.
OWASP LLM Top 10 - LLM01 Prompt Injection: https://genai.owasp.org/llmrisk/llm01-prompt-injection/	Grounding for treating webpage content as untrusted data in agent pipelines.